A FULL PRIOR WORK RESULT

Table 7 substitutes each prior method (SNAILS, Agentic Schema Refinement (ASR), TEMPLAR) to compare against their respective operators and keeps every other part of the pipeline unchanged. SNAILS and ASR fall below the baseline in every cell, and TEMPLAR is mixed.

Table 7: Each prior method substituted for the operator it anticipates, under BASESQL with top-k=3; EX with delta over the row's baseline. **: McNemar $p < 0.05$; *: $0.05 \leq p < 0.10$.

Backbone	Baseline	*renaming* +R	*renaming* SNAILS	*abstraction* +A	*abstraction* ASR	*partitioning* +P	*partitioning* TEMPLAR
BIRD-Union (n=767)							
GPT-4.1 mini	49.80	50.46 (+0.65)	49.02 (-0.78)	50.85 (+1.04)	49.67 (-0.13)	50.46 (+0.65)	50.20 (+0.39)
Gemini 2.5-flash-lite	48.11	48.63 (+0.52)	36.25 (-11.86)	47.98 (-0.13)	47.20 (-0.91)	50.72 (+2.61)**	46.41 (-1.69)
GPT-5.4 mini	52.80	52.93 (+0.13)	49.80 (-3.00)	55.54 (+2.74)**	52.54 (-0.26)	53.06 (+0.26)	52.54 (-0.26)
Spider-Union (n=502)							
GPT-4.1 mini	79.48	80.08 (+0.60)	78.29 (-1.20)	81.27 (+1.79)	79.28 (-0.20)	79.48 (0.00)	80.48 (+1.00)
Gemini 2.5-flash-lite	80.48	81.47 (+1.00)	78.09 (-2.39)	80.48 (0.00)	80.08 (-0.40)	81.67 (+1.20)	80.08 (-0.40)
GPT-5.4 mini	81.67	82.47 (+0.80)	81.08 (-0.60)	84.46 (+2.79)**	80.68 (-1.00)	82.47 (+0.80)	81.87 (+0.20)

B ROBUSTNESS AND WORKLOAD AVAILABILITY FULL RESULT

B.1 Across workload-history availability

Figure 11 and Figure 12 present how performance changes as the amount of available historical workload increases (0%, 50%, 100%). We evaluate these variations in design-time history across both zero-shot (no prompt-time history) and few-shot (with prompt-time history) settings. Configurations containing +P are omitted from zero-shot evaluation as partitioning requires design-time history to construct partitions, and applying it without retrieving the associated prompt-time demonstrations would be impractical. The results show two consistent trends. First, schema transformation remains beneficial even with 0% design-time history (when comparing the gray bars for optimizations like +A and +A+R with the baseline), confirming that the framework is effective without any historical workload. Second, performance generally improves as more historical workload data becomes available, indicating that workload-aware schema transformation and retrieval yield additional gains when such information is available. The conclusion is similar when we consider the BIRD-per-DB setting (the original BIRD benchmark but still no evidence or column description), where the main difference is that the gaps between the 50% and 100% design-time history settings are larger than on BIRD-Union (when comparing the dark blue bars with the red bars). Schema transformation is therefore not tied to having a workload history, functioning as a viable optimization even in completely history-free environments.

B.2 Pruning vs. demonstrations

Table 8 separates the two things +P changes: the schema the model sees (full / pruned schema) and the pool its demonstrations come from (global / partition-scoped top-k).

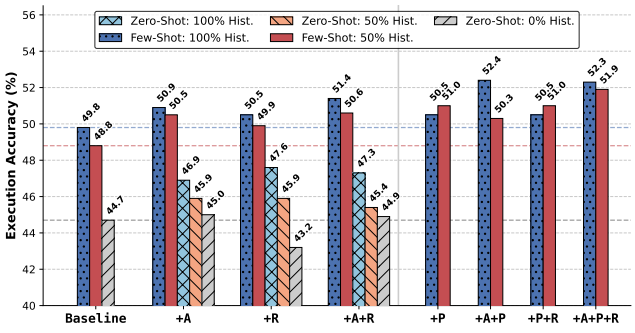


Figure 11: Execution accuracy on BIRD-Union under 0/50/100% historical workload availability by transformation.

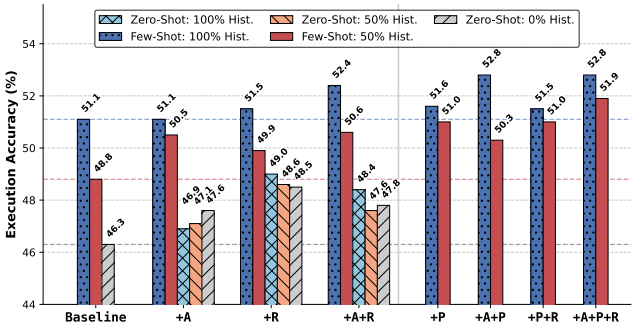


Figure 12: Per-database execution accuracy on BIRD under 0/50/100% historical workload availability by transformation.

Table 8: +P decomposed into schema pruning (rows) and demonstration scoping (columns), BIRD-Union. Global top-k means the few-shot demonstrations are retrieved from all available queries in the workload, while partition-scoped top-k restricts the retrieval to queries that only access the tables/columns in the pruned schema.

	Global top-k	Partition-scoped top-k
GPT-4.1 mini		
Full schema	49.80	*50.33* (+0.53)
Pruned schema	***50.98*** (+1.18)	*50.46* (+0.66)
Gemini 2.5-flash-lite		
Full schema	48.11	48.11 (+0.00)
Pruned schema	*49.41* (+1.30)	***50.72*** (+2.61)**

B.3 Template disjoint shift

To test whether the gains depend on similar queries in the history, we keep the historical workload fixed and evaluate only on test questions for which no historical query uses the same tables with the same SQL shape. **Skeleton** defines shape as the SQL with literals and identifiers masked (599 questions); **keyword** defines it as the set of SQL keywords (497 questions). Table 9 reports the deltas on both subsets and on the full test set.

Table 9: Deltas over the baseline on the full test set and on the two template-disjoint subsets, BIRD-Union. **: McNemar $p < 0.05$; *: $0.05 \leq p < 0.10$.

Backbone	Set	Baseline	+A	+P	+A+P	+R	+A+P+R
GPT-4.1 mini	Full (767)	49.80	(+1.04)	(+0.65)	(+2.61)**	(+0.65)	(+2.48)*
GPT-4.1 mini	Skeleton (599)	45.24	(+1.17)	(+1.00)	(+2.17)	(+2.17)	(+3.51)**
GPT-4.1 mini	Keyword (497)	44.27	(+0.60)	(+0.80)	(+2.01)	(+2.21)	(+2.01)
GPT-5.4 mini	Full (767)	52.80	(+2.74)**	(+0.26)	(+1.69)	(+0.13)	(+1.04)
GPT-5.4 mini	Skeleton (599)	47.58	(+2.17)	(+0.50)	(+1.34)	(+1.00)	(+1.50)
GPT-5.4 mini	Keyword (497)	46.88	(+2.82)**	(+0.80)	(+1.61)	(+0.80)	(+0.80)
Gemini 2.5-fl	Full (767)	48.11	(-0.13)	(+2.61)**	(+3.52)**	(+0.52)	(+4.17)**
Gemini 2.5-fl	Skeleton (599)	43.74	(-1.17)	(+2.00)	(+2.50)	(+1.00)	(+4.67)**
Gemini 2.5-fl	Keyword (497)	43.06	(-0.80)	(+2.21)	(+3.42)**	(+1.61)	(+4.63)**

B.4 Star Schema, Oracle View and Three-Table Views vs. +A

To compare against a design derived from the schema alone, we build an FK-derived star view set: for each eligible table, one view joining it with its FK-connected tables. Table 10 compares the two view sets on their structure (size, coverage, exact match with the gold table set, irrelevant tables offered) and on EX for three backbones.

Table 10: FK-derived star view set vs. workload-mined views on BIRD-Union, structure and EX on three backbones.

	Base	Star (FK)	Mined (+A)
Catalogue properties (schema-derived, backbone-independent)			
views	—	46	64
tables per view	—	2.7	2.3
table set *covers* gold	—	**88.9%**	87.1%
table set *equals* gold	—	40.3%	**58.8%**
irrelevant tables offered	—	0.86	**0.34**
gold re-expressible on views	—	**79.5%**	67.1%
Execution accuracy			
GPT-4.1 mini	49.80	48.63 (-1.17)	**50.85** (+1.04)
Gemini 2.5-flash-lite	**48.11**	43.29 (-4.82)	47.98 (-0.13)
GPT-5.4 mini	52.80	51.89 (-0.91)	**55.54** (+2.74)

Table 11 reports two further view sets. The *oracle* condition gives the model the single mined view whose table set exactly matches the question's, an upper bound for view selection. The *≥3-table* condition keeps only views over three or more tables. The upper rows give deltas over the baseline on all 767 questions. The lower rows give absolute EX on the 459 questions of the measurement subset (based on the oracle view set).

Table 11: View-set ablations on BIRD-Union, all mined views, three-or-more-table views only, and the oracle single view.

View pool	GPT-4.1 mini	GPT-5.4 mini	Gemini 2.5-fl
No view (baseline EX)	49.80	52.80	48.11
+A, all 64 mined views	*+1.04*	*+2.74*	−0.13
+A, ≥3-table views only	*+0.13*	*+1.04*	*+0.13*
+A, oracle single view	***+3.13***	***+5.09***	***+1.30***
Measurement subset, n=459 (absolute EX)			
No view	48.80	51.20	47.49
+A, all 64 mined views	49.67	54.47	46.41
+A, oracle single view	**54.03**	**59.69**	**49.67**

B.5 Human validation on Renaming

We manually inspected all generated renames and found three incorrect column renames (`RF`, `RA`, and `Description`), all in the `thrombosis_prediction` database. We corrected the three names and reran every renamed configuration on the 82 `thrombosis_prediction` questions with all three backbones. Table 12 reports EX for the baseline, the original renames (Ours), and the corrected renames.

Table 12: Effect of correcting the three wrong renames on every renamed configuration, `thrombosis_prediction` in BIRD-Union (n=82).

					Corrected vs.	
Backbone	Config	Baseline	Ours	Corrected	Ours	Baseline
GPT-4.1 mini	+R	23.17	26.83	24.39	(-2.44)	(+1.22)
GPT-4.1 mini	+A+R	23.17	23.17	23.17	(0.00)	(0.00)
GPT-4.1 mini	+P+R	23.17	21.95	21.95	(0.00)	(-1.22)
GPT-4.1 mini	+A+P+R	23.17	23.17	**25.61**	(+2.44)	(+2.44)
GPT-5.4 mini	+R	29.27	**30.49**	25.61	(-4.88)	(-3.66)
GPT-5.4 mini	+A+R	29.27	24.39	25.61	(+1.22)	(-3.66)
GPT-5.4 mini	+P+R	29.27	26.83	**28.05**	(+1.22)	(-1.22)
GPT-5.4 mini	+A+P+R	29.27	26.83	**28.05**	(+1.22)	(-1.22)
Gemini 2.5-fl	+R	18.29	20.73	**26.83**	(+6.10)	(+8.54)
Gemini 2.5-fl	+A+R	18.29	26.83	**26.83**	(0.00)	(+8.54)
Gemini 2.5-fl	+P+R	18.29	23.17	20.73	(-2.44)	(+2.44)
Gemini 2.5-fl	+A+P+R	18.29	21.95	23.17	(+1.22)	(+4.88)

B.6 Repeated runs

We repeat the complete `+A+P+R` configuration three times for every backbone and benchmark rather than rerunning every ablation. Across these settings, the run-to-run standard deviation is 0.2–0.8 EX percentage points (See Table 13 below).

Table 13: Mean ± std of EX over three independent runs of BASESQL +A+P+R; run 1 is the published one.

Benchmark	Backbone	run 1	run 2	run 3	mean ± std
BIRD-Union	GPT-4.1 mini	52.28	53.06	52.15	52.50 ± 0.49
BIRD-Union	GPT-5.4 mini	53.85	53.59	55.15	54.20 ± 0.84
BIRD-Union	Gemini 2.5-fl	52.28	52.67	52.67	52.54 ± 0.23
Spider-Union	GPT-4.1 mini	81.27	80.90	81.50	81.22 ± 0.30
Spider-Union	GPT-5.4 mini	84.26	85.06	84.66	84.66 ± 0.40
Spider-Union	Gemini 2.5-fl	83.07	83.67	83.67	83.47 ± 0.35

B.7 Error Propagation Full Result

An upstream operator can remove a table that a later operator needs. We check this for +P into +A: +P misses at least one gold table for 24 of the 767 questions. Table 14 splits the questions by whether the partition contains every gold table and reports EX for each configuration on both groups.

Table 14: EX breakdown by upstream error cases, BIRD-Union, GPT-4.1 mini (n=767).

Stratum	n	Baseline	+A	+P	+A+P
partition complete	743	50.6	51.5	51.7	**53.6**
partition *missing* a gold table	24	**25.0**	29.2	12.5	16.7

C PER-DOMAIN ANALYSIS AND BOUNDARY CONDITIONS

Table 15 and Figure 13 provide a database-level breakdown on BIRD-Union, evaluated using BaseSQL with Gemini 2.5-flash-lite. Overall, the transformed schema improves performance on 8 out of 11 databases, leaves one unchanged, and degrades performance on two. This shows that logical schema transformation is broadly useful, but not uniformly beneficial. The largest gains occur in domains where the dominant difficulty appears to be schema-level reasoning, such as weak identifier semantics, noisy schema context, or nontrivial relational access paths. For example, gains on domains such as `toxicology`, `student_club`, and `formula_1` suggest that schema abstraction and partitioning can help expose recurring access patterns and reduce irrelevant context.

Table 15: Database characteristics sorted by baseline performance. N_{1-3} denote the ratio of column identifier naturalness (N_1: most, N_3: least).

Database Domain	Tbls	Cols	N_1	N_2	N_3	FKs	Qrys
thrombosis_prediction	3	64	18.8%	4.7%	76.6%	2	82
california_schools	3	89	50.6%	44.9%	4.5%	2	47
financial	8	55	60.0%	12.7%	27.3%	8	46
european_football_2	7	199	51.3%	33.7%	15.1%	30	58
formula_1	13	94	88.3%	7.4%	4.3%	19	86
toxicology	4	11	72.7%	27.3%	0.0%	5	81
card_games	6	115	73.9%	25.2%	0.9%	4	93
debit_card_specializing	5	21	95.2%	4.8%	0.0%	2	34
student_club	8	48	100%	0.0%	0.0%	8	76
superhero	10	31	67.7%	32.3%	0.0%	11	59
codebase_community	8	71	88.7%	11.3%	0.0%	13	105

The negative cases reveal the boundary conditions of the approach. The two queries below illustrate the failure mode: the renamed schema's identifiers (e.g., `Bank_Clients`, `Energy_Stations`) are plausible aliases but point the model to the wrong table for the question, leading to a confidently wrong join.

```
Example A (financial). In the branch where the second-highest number of crimes were committed in 1995 occurred, how many male clients are there?
Q_corr = SELECT COUNT(T1.client_id) FROM client AS T1 WHERE T1.gender = 'M' AND T1.district_id = (SELECT T2.district_id FROM account AS T2 WHERE STRFTIME('%Y', T2.date) = '1995' GROUP BY T2.district_id ORDER BY COUNT(*) DESC LIMIT 1 OFFSET 1);

Q_wrong = SELECT COUNT(T1.client_id) FROM `Bank_Clients` AS T1 INNER JOIN `Bank_Districts` AS T2 ON T1.district_id = T2.district_id WHERE T2.district_id = (SELECT district_id FROM `Bank_Districts` ORDER BY `crime_count_95` DESC LIMIT 1 OFFSET 1) AND T1.gender = 'M';
```

```
Example B (debit_card_specializing). What is the nationality of the customer who spent 548.4 in 2012/8/24?
Q_corr = SELECT T2.Segment FROM transactions_1k AS T1 INNER JOIN customers AS T2 ON T1.CustomerID = T2.CustomerID WHERE T1.Date = '2012-08-24' AND T1.Price = 548.4;

Q_wrong = SELECT T2.country_code FROM Energy_Sales AS T1 INNER JOIN Energy_Stations AS T2 ON T1.station_id = T2.station_id WHERE T1.trans_date = '2012-08-24' AND T1.total_price = 548.4;
```

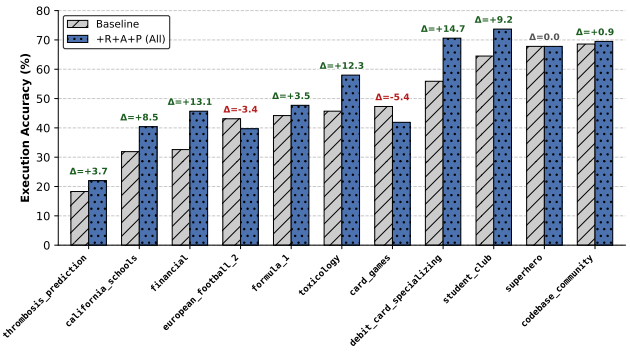


Figure 13: Database-level breakdown of execution accuracy on BIRD-Union: baseline vs. the full +A+P+R transformation.

When errors are driven less by schema structure and more by domain-specific value grounding, subtle predicate interpretation, or fine-grained column selection, schema transformation may provide limited benefit or even hurt. For example, on `thrombosis_prediction`, performance remains unchanged despite a high proportion of complex queries and less natural schema identifiers, suggesting that the remaining errors may be dominated by medical terminology, value grounding, or predicate semantics rather than by schema transformation alone. In such cases, abstraction may introduce unhelpful alternatives, partitioning may prune useful context, or renaming may remove lexical cues present in the original schema. The largest degradation occurs on `card_games` ($\Delta = -5.4$), a wide schema with many semantically dense attributes but relatively low average join complexity. These results refine the operator-level conclusion: Text-to-SQL-friendly logical design is useful but non-monotonic, and no single transformation is uniformly optimal across schemas.

Performance breakdown by query complexity. Figure 14 reports performance breakdowns on BIRD-Union by required joins and by the query difficulty labels provided by the BIRD benchmark. This diagnostic analysis decomposes the same experimental results evaluated in Figure 13. The goal is to understand where the complete schema-optimization pipeline improves over the baseline. The results show that schema transformation improves both single-table and join-requiring queries, with slightly larger gains

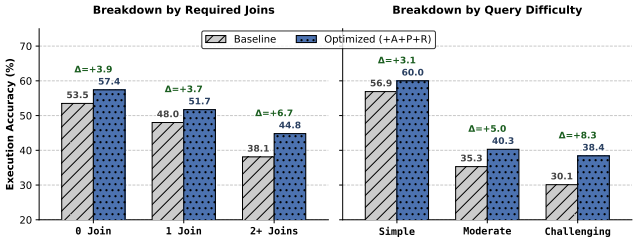


Figure 14: Performance breakdown by joins and difficulty

for join-requiring queries. The effect is more pronounced in the difficulty-based breakdown: gains increase from simple to moderate, then to challenging queries. This suggests that logical schema transformation is especially helpful when the model faces a greater schema-induced reasoning burden.

D ENTERPRISE BENCHMARK FULL RESULT

BEAVER is an enterprise data-warehouse benchmark with three databases. Due to computational and time constraints, we used the full `neutron` and `nova`, and sampled 500/500 for historical and test questions from `dw`, totalling 1,534 historical queries and 1,536 test questions. Table 16 reports EX for each database separately and pooled; the paper prints only the pooled column.

Table 16: BEAVER per database and pooled, BaseSQL with GPT-4.1 mini, top-k=10; the paper prints the pooled row. **: McNemar $p < 0.05$; *: $0.05 \le p < 0.10$.

Arm	dw (n=500)	neutron (n=509)	nova (n=527)	pooled (n=1,536)
Baseline	7.8	33.2	22.6	21.3
+A	7.4 (-0.4)	34.2 (+1.0)	23.3 (+0.8)	21.7 (+0.5)
+P	8.6 (+0.8)	33.0 (-0.2)	24.1 (+1.5)	22.0 (+0.7)
+R	8.8 (+1.0)	34.0 (+0.8)	21.6 (-0.9)	21.5 (+0.3)
+A+P	9.0 (+1.2)	34.0 (+0.8)	**25.4** (+2.8)**	**22.9** (+1.6)**
+A+R	**10.6** (+2.8)**	34.0 (+0.8)	23.3 (+0.8)	22.7 (+1.4)*
+P+R	9.2 (+1.4)	**35.0** (+1.8)	22.8 (+0.2)	22.4 (+1.1)
+A+P+R	**10.6** (+2.8)**	34.6 (+1.4)	23.0 (+0.4)	22.8 (+1.5)*

E COST OF TRANSFORMATION

Offline cost. Table 17 breaks down the LLM calls and tokens needed to transform BIRD-Union once (`+R+A+P`, 75 tables, 767 historical queries), by phase: view-DDL synthesis, view-SQL rewriting, history rewriting, and rename synthesis.

Online cost. Table 18 reports prompt tokens per question for every configuration, split across schema linking, SQL generation, and SQL revision (BaseSQL, GPT-4.1 mini, BIRD-Union). Its italic rows give the measured LLM time per question for the baseline, `+A`, and `+A+P+R`.

BEAVER. Table 19 reports prompt tokens per question on BEAVER on BaseSQL, for each database.

Table 17: Offline cost of one transformation of BIRD-Union (75 tables, 767 historical queries).

Phase	calls	input tok.	output tok.
View DDL synthesis	93	185,029	61,594
View-SQL rewrite	6	221,654	57,115
History rewrite	6	126,118	56,756
Rename synthesis (initial)	1	76,715	9,614
Rename synthesis (retry)	1	57,743	1,058
Total	**107**	**667,259**	**186,137**

Table 18: Prompt tokens and measured latency per question on BIRD-Union with GPT-4.1 mini. Latency is measured for the baseline, +A, +A+P+R.

Config	stage 1	stage 2	stage 3	total	Δ vs base
Baseline	23,315	21,898	21,982	67.2K	—
latency	*2.47 s*	*2.20 s*	*2.43 s*	*7.10 s*	—
+R	22,284	22,467	22,561	67.3K	(+0.2%)
+A	23,315	25,495	25,581	74.4K	(+10.7%)
latency	*2.35 s*	*2.23 s*	*2.54 s*	*7.12 s*	(+0.3%)
+P	23,315	3,073	3,162	29.6K	(-56.0%)
+R+A	22,284	26,466	26,565	75.3K	(+12.1%)
+R+P	22,284	3,186	3,279	**28.7K**	(-57.2%)
+A+P	23,315	6,670	6,754	36.7K	(-45.3%)
+R+A+P	22,284	7,186	7,278	36.7K	(-45.3%)
latency	*2.31 s*	*2.39 s*	*2.39 s*	*7.09 s*	(-0.1%)

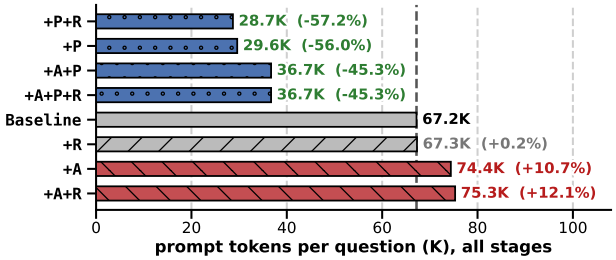


Figure 15: Prompt tokens per question, all stages (BaseSQL, GPT-4.1 mini, BIRD-Union).

Table 19: BEAVER prompt tokens per question on BaseSQL, with the change over the baseline.

	dw	neutron	nova
Baseline	159.1K	136.9K	138.7K
+A	183.3K (+15%)	150.8K (+10%)	188.4K (+36%)
+P	102.7K (-35%)	67.3K (-51%)	67.8K (-51%)
+R	136.5K (-14%)	99.6K (-27%)	129.5K (-7%)
+A+P	126.4K (-21%)	82.4K (-40%)	117.0K (-16%)
+A+R	158.9K (-0.1%)	115.1K (-16%)	179.2K (+29%)
+P+R	90.0K (-43%)	54.1K (-60%)	65.5K (-53%)
+A+P+R	112.6K (-29%)	69.6K (-49%)	114.7K (-17%)

F STATISTICAL SIGNIFICANCE FULL RESULT

Table 20 gives McNemar's exact p for every configuration, each against its row's baseline. Table 21 gives the 95% confidence interval of each paired accuracy difference, computed from the same gains (c) and losses (b) as the test: $\hat{\Delta} = (c - b)/n$ and $\mathrm{SE} = \sqrt{b + c - (c - b)^2/n}\,/\,n$.

Table 20: McNemar exact p for every configuration of Table 2, each against its row's baseline. **: $p < 0.05$; *: $0.05 \le p < 0.10$.

Approach	+A	+P	+R	+A+P	+A+R	+P+R	+A+P+R
BIRD-Union (n=767)							
BaseSQL	.358	.653	.699	.042**	.290	.702	.081*
BaseSQL (50% Hist.)	.111	.068*	.452	.241	.223	.149	.045**
DIN-SQL	.358	<.001**	.081*	.029**	.241	.006**	.004**
MAC-SQL	.343	.097*	.755	.040**	.057*	.078*	.046**
CSC-SQL (Qwen2.5-Coder-7B)	.826	.224	.391	1.000	.501	.592	.723
AutoLink (DeepSeek-V3/R1)	<.001**	<.001**	.002**	.031**	<.001**	.001**	<.001**
BaseSQL on other backbones							
BaseSQL (Gemini 2.5-flash-lite)	1.000	.024**	.777	.005**	.087*	.024**	.004**
BaseSQL (GPT-5.4 mini)	.005**	.897	1.000	.160	.218	.426	.461
Spider-Union (n=502)							
BaseSQL	.136	1.000	.761	.856	1.000	.885	.262
BaseSQL (50% Hist.)	.096*	.523	1.000	1.000	.749	.533	.337
DIN-SQL	.189	.839	.060*	.200	.678	.001**	.085*
MAC-SQL	.500	.533	.065*	.533	.014**	.039**	.066*
CSC-SQL (Qwen2.5-Coder-7B)	.281	.839	.222	.281	.471	.878	.253
AutoLink (DeepSeek-V3/R1)	.029**	.458	.230	.458	.143	.243	.473
BaseSQL on other backbones							
BaseSQL (Gemini 2.5-flash-lite)	1.000	.471	.615	.560	.683	.148	.098*
BaseSQL (GPT-5.4 mini)	.024**	.644	.636	.749	.081*	.054*	.066*

Table 21: Paired accuracy difference (points) with its 95% confidence interval for every configuration of Table 2, except the 50%-history rows. **: McNemar $p < 0.05$; *: $0.05 \le p < 0.10$.

Approach	+A	+P	+R	+A+P	+A+R	+P+R	+A+P+R
BIRD-Union (n=767)							
BaseSQL	+1.0 [−0.9, +3.0]	+0.7 [−1.6, +2.9]	+0.7 [−2.0, +3.3]	+2.6 [+0.2, +5.0]**	+1.6 [−1.1, +4.2]	+0.7 [−2.0, +3.3]	+2.5 [−0.2, +5.1]*
DIN-SQL	+1.0 [−0.9, +3.0]	+3.9 [+1.8, +6.0]**	+2.5 [−0.2, +5.1]*	+2.5 [+0.4, +4.6]**	+1.7 [−0.9, +4.3]	+4.0 [+1.2, +6.8]**	+4.0 [+1.4, +6.7]**
MAC-SQL	+1.3 [−1.1, +3.7]	+2.1 [−0.2, +4.4]*	+0.5 [−1.9, +3.0]	+2.6 [+0.2, +5.0]**	+2.6 [+0.1, +5.2]*	+2.5 [−0.1, +5.1]*	+2.7 [+0.2, +5.3]**
CSC-SQL (Qwen2.5-Coder-7B)	+0.4 [−1.9, +2.7]	+1.6 [−0.7, +3.9]	+1.3 [−1.4, +4.0]	+0.0 [−2.4, +2.4]	+1.0 [−1.6, +3.7]	+0.9 [−1.9, +3.7]	+0.7 [−2.2, +3.5]
AutoLink (DeepSeek-V3/R1)	+5.6 [+3.1, +8.1]**	+4.8 [+2.3, +7.3]**	+4.3 [+1.7, +7.0]**	+2.7 [+0.4, +5.1]**	+5.7 [+3.1, +8.4]**	+4.6 [+1.8, +7.3]**	+4.7 [+2.0, +7.4]**
BaseSQL on other backbones							
BaseSQL (Gemini 2.5-flash-lite)	−0.1 [−2.3, +2.0]	+2.6 [+0.4, +4.8]**	+0.5 [−2.2, +3.2]	+3.5 [+1.2, +5.9]**	+2.5 [−0.2, +5.2]*	+3.3 [+0.6, +6.0]**	+4.2 [+1.4, +6.9]**
BaseSQL (GPT-5.4 mini)	+2.7 [+0.9, +4.6]**	+0.3 [−1.7, +2.2]	+0.1 [−2.4, +2.6]	+1.7 [−0.5, +3.9]	+1.7 [−0.8, +4.2]	+1.2 [−1.4, +3.7]	+1.0 [−1.4, +3.5]
Spider-Union (n=502)							
BaseSQL	+1.8 [−0.3, +3.9]	+0.0 [−1.8, +1.8]	+0.6 [−2.0, +3.2]	+0.4 [−1.7, +2.5]	+0.2 [−2.7, +3.1]	+0.4 [−2.3, +3.1]	+1.8 [−1.0, +4.6]
DIN-SQL	+1.4 [−0.4, +3.2]	+0.4 [−1.5, +2.3]	+2.6 [+0.1, +5.1]*	+1.6 [−0.5, +3.7]	+0.8 [−2.0, +3.6]	+4.2 [+1.7, +6.7]**	+2.6 [−0.1, +5.3]*
MAC-SQL	+1.0 [−1.3, +3.3]	+1.0 [−1.5, +3.5]	+2.4 [+0.1, +4.7]*	+1.0 [−1.5, +3.5]	+3.2 [+0.8, +5.6]**	+2.8 [+0.3, +5.2]**	+2.6 [+0.0, +5.1]*
CSC-SQL (Qwen2.5-Coder-7B)	+1.4 [−0.8, +3.6]	+0.4 [−1.5, +2.3]	+1.8 [−0.8, +4.3]	+1.4 [−0.8, +3.6]	+1.2 [−1.5, +3.9]	+0.4 [−2.1, +2.9]	+1.8 [−0.9, +4.5]
AutoLink (DeepSeek-V3/R1)	+2.6 [+0.4, +4.8]**	+1.0 [−1.1, +3.1]	+1.6 [−0.7, +3.9]	+1.0 [−1.1, +3.1]	+2.0 [−0.4, +4.4]	+1.6 [−0.7, +3.9]	+1.0 [−1.2, +3.2]
BaseSQL on other backbones							
BaseSQL (Gemini 2.5-flash-lite)	+0.0 [−2.3, +2.3]	+1.2 [−1.5, +3.9]	+1.0 [−2.1, +4.1]	+1.0 [−1.7, +3.7]	+0.8 [−2.1, +3.7]	+2.4 [−0.6, +5.4]	+2.6 [−0.2, +5.4]*
BaseSQL (GPT-5.4 mini)	+2.8 [+0.5, +5.1]**	+0.8 [−1.7, +3.3]	+0.8 [−1.7, +3.3]	+0.6 [−1.8, +3.0]	+2.4 [−0.1, +4.9]*	+2.8 [+0.2, +5.4]*	+2.6 [+0.0, +5.1]*